PES-VCS Lab Report

Name: Mebin J Thattil SRN: PES2UG24CS273 Repository: https://github.com/mebinthattil/PES2UG24CS273-pes-vcs

Overview

This report documents the implementation and verification of PES-VCS, a simplified local version control system built from the provided template repository. The required phases were completed by implementing:

- object.c: object storage and integrity verification
- tree.c: tree construction from the index
- index.c: staging area load/save/add operations
- commit.c: commit creation from the staged snapshot

The project was built with make all, verified with the provided unit tests, validated with manual command flows for Phases 3 and 4, and checked again with make test-integration.

Phase 1

Screenshot 1A: ./test_objects

```
mebin@mebin-QEMU-Virtual-Machine:~/Downloads/OS-Orange/PES2UG24CS273-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1A

Screenshot 1B: find .pes/objects -type f

```
mebin@mebin-QEMU-Virtual-Machine:~/Downloads/OS-Orange/PES2UG24CS273-pes-vcs$ find .pes/objects -type f | sort
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

Screenshot 1B

Phase 2

Screenshot 2A: ./test_tree

```
mebin@mebin-QEMU-Virtual-Machine:~/Downloads/OS-Orange/PES2UG24CS273-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2A

Screenshot 2B: Raw tree object with xxd

```
mebin@mebin-QEMU-Virtual-Machine:/tmp/pes-treecap-session$ xxd "$tree_file" | head -20
00000000: 7472 6565 2034 3600 3130 3036 3434 206d  tree 46.100644 m
00000010: 6169 6e2e 6300 fd8a 0f23 eb49 4c98 e85b  ain.c....#.IL..[
00000020: 3819 a691 d2aa 6a5c c7ff 2d65 263f cf93  8.....j\..-e&?..
00000030: f35c 8639 d7c7                           .\.9..
```

Screenshot 2B

Phase 3

Screenshot 3A: pes init → pes add → pes status

```
mebin@mebin-QEMU-Virtual-Machine:/tmp/pes-phase3cap-Tr3DVX$ pes init
Initialized empty PES repository in .pes/

mebin@mebin-QEMU-Virtual-Machine:/tmp/pes-phase3cap-Tr3DVX$ printf "hello\n" > file1.txt
mebin@mebin-QEMU-Virtual-Machine:/tmp/pes-phase3cap-Tr3DVX$ printf "world\n" > file2.txt
mebin@mebin-QEMU-Virtual-Machine:/tmp/pes-phase3cap-Tr3DVX$ pes add file1.txt file2.txt

mebin@mebin-QEMU-Virtual-Machine:/tmp/pes-phase3cap-Tr3DVX$ pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)
```

Screenshot 3A

Screenshot 3B: .pes/index

```
mebin@mebin-QEMU-Virtual-Machine:/tmp/pes-phase3cap-Tr3DVX$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776746439 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776746439 6 file2.txt
```

Screenshot 3B

Phase 4

Screenshot 4A: pes log

```
mebin@mebin-QEMU-Virtual-Machine:/tmp/pes-phase4cap-bku1Ap$ ./pes log
commit eb2ee4f86931e44c5450e06dba9e272ae0af2bb26ed8b4fa945943ad1f4328c5
Author: Mebin J Thattil <PES2UG24CS273>
Date:   1776746451

    Add farewell

commit 957f4c4d585d03643483b85c326dac220f6e7c5a3018a8cd54e43d77d3fda50e
Author: Mebin J Thattil <PES2UG24CS273>
Date:   1776746451

    Add world

commit 519c0834054f9ef6829e3b8a093d9cfc28c844c99969d83a59fc415f838cf383
Author: Mebin J Thattil <PES2UG24CS273>
Date:   1776746451

    Initial commit
```

Screenshot 4A

Screenshot 4B: find .pes -type f | sort

```
mebin@mebin-QEMU-Virtual-Machine:/tmp/pes-phase4cap-bku1Ap$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/51/9c0834054f9ef6829e3b8a093d9cfc28c844c99969d83a59fc415f838cf383
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/95/7f4c4d585d03643483b85c326dac220f6e7c5a3018a8cd54e43d77d3fda50e
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/eb/2ee4f86931e44c5450e06dba9e272ae0af2bb26ed8b4fa945943ad1f4328c5
.pes/refs/heads/main
```

Screenshot 4B

Screenshot 4C: cat .pes/refs/heads/main and cat .pes/HEAD

```
mebin@mebin-QEMU-Virtual-Machine:/tmp/pes-phase4cap-bku1Ap$ cat .pes/refs/heads/main
eb2ee4f86931e44c5450e06dba9e272ae0af2bb26ed8b4fa945943ad1f4328c5

mebin@mebin-QEMU-Virtual-Machine:/tmp/pes-phase4cap-bku1Ap$ cat .pes/HEAD
ref: refs/heads/main
```

Screenshot 4C

Final Integration Test

make test-integration

Final Integration Test

Analysis Answers

Q5.1 Branching and Checkout

To implement pes checkout <branch>, the system would first resolve the target branch file at .pes/refs/heads/<branch> and read the commit hash stored in it. Then HEAD would be updated to ref: refs/heads/<branch> so future commits advance that branch. After that, the target commit would be read, its root tree would be traversed recursively, and the working directory would be rewritten to match that tree exactly. Existing tracked files that are not present in the target tree would need to be removed, files with changed blob hashes would need to be overwritten, executable bits would need to be restored from tree modes, and the index would need to be rewritten so it matches the checked-out snapshot.

The operation is complex because checkout affects both metadata and real filesystem contents. It is not just changing HEAD; it must safely replace the working directory, preserve permissions, handle nested directories, detect conflicts with local edits, and avoid partially switching the repository if an error happens midway through the process.

Q5.2 Dirty Working Directory Conflict Detection

The index already stores each tracked file's path, blob hash, modification time, and size. To detect checkout conflicts, I would compare three states for every tracked path that differs between the current branch and the target branch:

1. The blob hash in the current HEAD tree.
2. The blob hash in the target branch's tree.
3. The current working-directory file compared against the index metadata and, when needed, the staged blob hash.

If the index says the file should have a specific size and mtime, but stat() on the working copy shows different metadata, that is a likely local edit and should trigger a deeper check by hashing the working copy and comparing it with the index's staged blob hash. If the working copy differs from the index, and the target branch also wants a different version of that same tracked path, checkout must refuse because switching would overwrite uncommitted user changes. This uses the index as the expected clean snapshot and the object store as the source of truth for the staged and committed contents.

Q5.3 Detached HEAD

In detached HEAD state, HEAD contains a commit hash directly rather than a symbolic branch reference. New commits still work, but each new commit only updates HEAD itself, not any branch file in .pes/refs/heads/. That means the commits exist in the object store and remain reachable only through the temporary detached HEAD position.

If the user later checks out a normal branch, those detached commits can become unreachable unless a new branch or tag is created that points to them first. Recovery is possible as long as the commit hashes are still known and the objects have not been garbage-collected. A user could recover them by creating a new branch reference file containing the detached commit hash, or by manually setting HEAD or another ref back to that commit chain.

Q6.1 Garbage Collection and Space Reclamation

The standard algorithm is a mark-and-sweep traversal:

1. Start from every live reference, such as all files under .pes/refs/heads/ and possibly HEAD if it stores a detached commit hash.
2. For each referenced commit, mark that commit as reachable.
3. From each reachable commit, mark its tree and parent commit, then recursively walk every tree entry.
4. For each tree entry, mark the referenced blob or subtree.
5. After traversal, scan .pes/objects/*/* and delete any object whose hash is not in the reachable set.

The right data structure for the reachable set is a hash set keyed by object hash, because membership checks must be fast and repeated often. In a repository with 100,000 commits and 50 branches, the traversal would visit every commit reachable from those 50 tips, but shared history means the number of unique commits visited would still usually be close to the total history size, not 50 times larger. So a realistic upper bound is on the order of 100,000 commit objects plus the trees and blobs reachable from those commits. The exact count depends on repository size, but the key point is that each unique object is visited once if the reachable set is used correctly.

Q6.2 GC Races with Concurrent Commits

Garbage collection is dangerous during a concurrent commit because a commit is created in stages. A new blob may be written first, then a tree that references it, then a commit that references the tree, and only at the end is the branch reference updated. If GC scans references during the gap before the branch file is updated, it may decide those freshly written objects are unreachable and delete them. The concurrent commit would then publish a ref to a commit whose tree or blobs are already missing, corrupting the repository.

Git avoids this kind of race by being conservative about object lifetime. Real Git uses additional references, reflogs, grace periods, and locking so that objects are not collected immediately after becoming temporarily unreachable. It also writes objects before updating refs atomically, and GC generally treats recently created loose objects as protected long enough that concurrent operations can finish publishing the references that make them reachable.